

LABSHEET - 5**NITIN PANWAR****BTECH(CSE) 3RD YEAR****CU240251575****SECTION - B****ROLL NO. - 39**

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure
```

```
# EXPERIMENT 5
# Feature Extraction and Image Analysis using SIFT and HOG
# =====

print("=" * 60)
print("EXPERIMENT 5")
print("Feature Extraction using SIFT and HOG")
print("=" * 60)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
EXPERIMENT 5
Feature Extraction using SIFT and HOG
=====
Himani Chaudhary
CU240250967
```

```
# 1. LOAD IMAGE
# =====

# Use r before the path to avoid Windows backslash problems
image_path = "/content/beautiful-nature-mountain-scenery-with-flowers-picjumbo-com.jpg"
# Load image
image = cv2.imread(image_path)

# Check whether image was loaded successfully
if image is None:
    print("\nERROR: Image not found!")
    print("Please check that the image path is correct.")
    print("\nPath used:")
    print(image_path)
    raise SystemExit

print("\nImage loaded successfully!")

# Display image size
print("Image size:", image.shape)

# Convert BGR to RGB for Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Grayscale conversion completed.")
print("Himani Chaudhary")
print("CU240250967")
```

```
Image loaded successfully!
Image size: (3584, 5376, 3)
Grayscale conversion completed.
Himani Chaudhary
CU240250967
```

```
# 2. DISPLAY ORIGINAL AND GRAYSCALE IMAGE
# =====

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.tight_layout()
plt.show()
print("Himani Chaudhary")
print("CU240250967")
```

Original Image



Grayscale Image



Himani Chaudhary
CU240250967

```
# 3. SIFT FEATURE EXTRACTION
# =====

print("\n" + "=" * 60)
print("SIFT FEATURE EXTRACTION")
print("=" * 60)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)

# Display number of keypoints
print("Number of SIFT keypoints:", len(keypoints))

# Check descriptors
if descriptors is not None:
    print("SIFT descriptor shape:", descriptors.shape)
else:
    print("No SIFT descriptors found.")
    print("Himani Chaudhary")
    print("CU240250967")
```

```
=====
SIFT FEATURE EXTRACTION
=====
Number of SIFT keypoints: 102076
SIFT descriptor shape: (102076, 128)
```

```
# 4. VISUALIZE SIFT KEYPOINTS
# =====
```

```

# Draw SIFT keypoints
sift_image = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

# Display SIFT keypoints
plt.figure(figsize=(12, 8))
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")
plt.tight_layout()
plt.show()
print("Himani Chaudhary")
print("CU240250967")

```

SIFT Keypoints



Himani Chaudhary
CU240250967

```

# 5. HOG FEATURE EXTRACTION
# =====

print("\n" + "=" * 60)
print("HOG FEATURE EXTRACTION")
print("=" * 60)

# Resize grayscale image for HOG
hog_image_input = cv2.resize(
    gray,
    (128, 128),
    interpolation=cv2.INTER_AREA
)

# Extract HOG features
hog_features, hog_image = hog(
    hog_image_input,
    orientations=9,

```

```

pixels_per_cell=(8, 8),
cells_per_block=(2, 2),
block_norm="L2-Hys",
visualize=True
)

# Display HOG feature information
print("Number of HOG features:", len(hog_features))
print("HOG feature extraction completed.")
print("Himani Chaudhary")
print("CU240250967")

```

```

=====
HOG FEATURE EXTRACTION
=====
Number of HOG features: 8100
HOG feature extraction completed.
Himani Chaudhary
CU240250967

```

```

# 6. VISUALIZE HOG
# =====

# Rescale HOG image for better visualization
hog_image_rescaled = exposure.rescale_intensity(
    hog_image,
    in_range=(0, 10)
)

# Display HOG input and visualization
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(hog_image_input, cmap="gray")
plt.title("Input Image for HOG")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_image_rescaled, cmap="gray")
plt.title("HOG Visualization")
plt.axis("off")

plt.tight_layout()
plt.show()
print("Himani Chaudhary")
print("CU240250967")

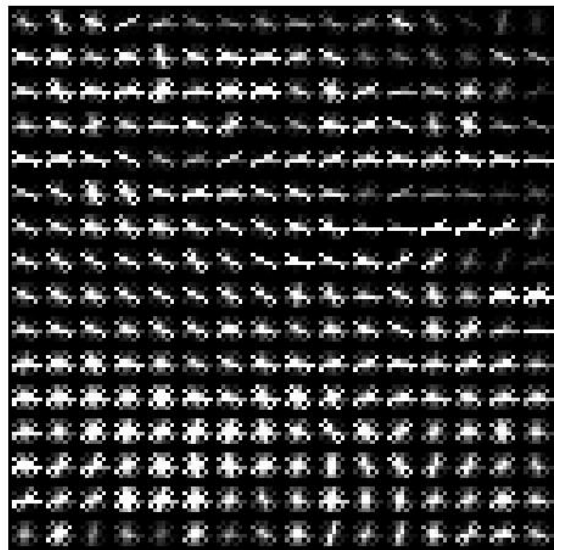
```

Input Image for HOG



Himani Chaudhary
CU240250967

HOG Visualization



```

# 7. DISPLAY FEATURE INFORMATION
# =====

```

```

print("\n" + "=" * 60)
print("FEATURE INFORMATION")
print("=" * 60)

print("PRIYANSHU SHARMA CU240250980")

print("\nSIFT:")
print("Keypoints detected:", len(keypoints))

if descriptors is not None:
    print("Descriptor dimensions:", descriptors.shape)
else:
    print("Descriptor dimensions: Not available")

print("\nHOG:")
print("Feature vector length:", len(hog_features))
print("Orientations:", 9)
print("Pixels per cell:", (8, 8))
print("Cells per block:", (2, 2))
print("Block normalization: L2-Hys")
print("Himani Chaudhary")
print("CU240250967")

```

```

=====
FEATURE INFORMATION
=====
PRIYANSHU SHARMA CU240250980

SIFT:
Keypoints detected: 102076
Descriptor dimensions: (102076, 128)

HOG:
Feature vector length: 8100
Orientations: 9
Pixels per cell: (8, 8)
Cells per block: (2, 2)
Block normalization: L2-Hys
Himani Chaudhary
CU240250967

```

```

# 8. CREATE SECOND IMAGE FOR SIFT MATCHING
# =====

print("\n" + "=" * 60)
print("SIFT IMAGE MATCHING")
print("=" * 60)

# Create second image by resizing the original image
image2 = cv2.resize(
    image,
    None,
    fx=0.9,
    fy=0.9,
    interpolation=cv2.INTER_LINEAR
)

# Convert second image to grayscale
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

# Detect SIFT features in second image
keypoints2, descriptors2 = sift.detectAndCompute(
    gray2,
    None
)

print("Keypoints in Image 1:", len(keypoints))
print("Keypoints in Image 2:", len(keypoints2))
print("Himani Chaudhary")
print("CU240250967")

```

```

=====
SIFT IMAGE MATCHING
=====
Keypoints in Image 1: 102076
Keypoints in Image 2: 90387
Himani Chaudhary
CU240250967

```

```
# 9. MATCH SIFT FEATURES USING FLANN
# =====

if descriptors is not None and descriptors2 is not None:

    print("\nPerforming SIFT feature matching...")

    # FLANN parameters
    index_params = dict(
        algorithm=1,    # KD-tree
        trees=5
    )

    search_params = dict(
        checks=50
    )

    # Create FLANN matcher
    flann = cv2.FlannBasedMatcher(
        index_params,
        search_params
    )

    # Find two nearest matches
    matches = flann.knnMatch(
        descriptors,
        descriptors2,
        k=2
    )

    # Lowe's ratio test
    good_matches = []

    for pair in matches:

        if len(pair) == 2:

            m, n = pair

            if m.distance < 0.7 * n.distance:
                good_matches.append(m)

    print("Total matches checked:", len(matches))
    print("Good matches found:", len(good_matches))
```

```
Performing SIFT feature matching...
Total matches checked: 102076
Good matches found: 53311
```

```
# =====
# 10. DISPLAY SIFT MATCHING RESULT
# =====

if image2 is not None and keypoints is not None and keypoints2 is not None:

    # Draw good matches using ORIGINAL BGR images
    matching_result = cv2.drawMatches(
        image,
        keypoints,
        image2,
        keypoints2,
        good_matches,
        None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    # Convert final result from BGR to RGB
    matching_result_rgb = cv2.cvtColor(
        matching_result,
        cv2.COLOR_BGR2RGB
    )

    # Display matching result
    plt.figure(figsize=(16, 8))
    plt.imshow(matching_result_rgb)
    plt.title("SIFT Image Matching")
    plt.axis("off")
    plt.tight_layout()
    plt.show()

else:
```

```
print("Image matching could not be performed.")
print("SIFT descriptors were not available.")
```

SIFT Image Matching



```
# 11. FINAL COMPARISON
# =====

print("\n" + "=" * 60)
print("SIFT vs HOG COMPARISON")
print("=" * 60)

print("""
SIFT:
- Detects important keypoints in an image.
- Generates feature descriptors for keypoints.
- Robust to scale changes.
- Robust to rotation changes.
- Useful for image matching and object recognition.
- Usually more computationally expensive than HOG.

HOG:
- Represents an image using gradient orientations.
- Captures shape and edge information.
- Commonly used for object detection.
- Computationally simpler than SIFT in many applications.
- Less suitable for rotation changes.
- Depends on parameters such as cell size and block size.
""")
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
SIFT vs HOG COMPARISON
=====

SIFT:
- Detects important keypoints in an image.
- Generates feature descriptors for keypoints.
- Robust to scale changes.
- Robust to rotation changes.
- Useful for image matching and object recognition.
- Usually more computationally expensive than HOG.

HOG:
- Represents an image using gradient orientations.
- Captures shape and edge information.
- Commonly used for object detection.
- Computationally simpler than SIFT in many applications.
- Less suitable for rotation changes.
- Depends on parameters such as cell size and block size.
```

Himani Chaudhary
CU240250967

```
# 12. OBSERVATION
# =====

print("\n" + "=" * 60)
print("OBSERVATION")
print("=" * 60)

print("""
SIFT successfully detected distinctive keypoints and generated
descriptors from the image.

HOG successfully extracted gradient-based features representing
the shape and edge structure of the image.

SIFT is suitable for image matching and recognition where
scale and rotation changes may occur.

HOG is suitable for shape-based object detection and
classification tasks.

Therefore, both techniques are useful handcrafted feature
extraction methods, but they are suitable for different
computer vision applications.
""")
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
OBSERVATION
=====

SIFT successfully detected distinctive keypoints and generated
descriptors from the image.

HOG successfully extracted gradient-based features representing
the shape and edge structure of the image.

SIFT is suitable for image matching and recognition where
scale and rotation changes may occur.

HOG is suitable for shape-based object detection and
classification tasks.

Therefore, both techniques are useful handcrafted feature
extraction methods, but they are suitable for different
computer vision applications.

Himani Chaudhary
CU240250967
```

```
# 13. EXPERIMENT SUMMARY
# =====

print("\n" + "=" * 60)
print("EXPERIMENT SUMMARY")
print("=" * 60)

print("Student Name: PRIYANSHU SHARMA")
print("Roll Number: CU240250980")

print("\nTechniques Used:")
print("1. SIFT - Scale Invariant Feature Transform")
print("2. HOG - Histogram of Oriented Gradients")
print("3. FLANN - Fast Library for Approximate Nearest Neighbors")
print("4. Lowe's Ratio Test")

print("\nResults:")
print("SIFT Keypoints:", len(keypoints))
print("HOG Feature Length:", len(hog_features))

if descriptors is not None:
    print("SIFT Descriptor Shape:", descriptors.shape)

if descriptors is not None and descriptors2 is not None:
    print("Good SIFT Matches:", len(good_matches))

print("\nConclusion:")
print("""
```

```
SIFT and HOG successfully extracted useful features from the
input image. SIFT provided local keypoints and descriptors
suitable for image matching, while HOG represented the image
using gradient and edge information suitable for shape-based
analysis and object detection.
""")
```

```
# =====
# 14. END OF EXPERIMENT
# =====
```

```
print("=" * 60)
print("EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 60)
print("Himani Chaudhary")
print("CU240250967")
```

```
=====
EXPERIMENT SUMMARY
=====
```

```
Student Name: PRIYANSHU SHARMA
Roll Number: CU240250980
```

```
Techniques Used:
```

1. SIFT - Scale Invariant Feature Transform
2. HOG - Histogram of Oriented Gradients
3. FLANN - Fast Library for Approximate Nearest Neighbors
4. Lowe's Ratio Test

```
Results:
```

```
SIFT Keypoints: 102076
HOG Feature Length: 8100
SIFT Descriptor Shape: (102076, 128)
Good SIFT Matches: 53311
```

```
Conclusion:
```

```
SIFT and HOG successfully extracted useful features from the
input image. SIFT provided local keypoints and descriptors
suitable for image matching, while HOG represented the image
using gradient and edge information suitable for shape-based
analysis and object detection.
```

```
=====
EXPERIMENT COMPLETED SUCCESSFULLY
=====
```

```
Himani Chaudhary
CU240250967
```